Ben Bubnick
Interview STAR Story Bank

Version 1.3 | Updated October 27, 2026 | 127 stories

This is a searchable interview reference. Use the accomplishment as the prompt, then scan the highlighted **S**, **T**, **A**, and **R** rows. Use the role index or PDF search to move directly to the relevant story.

Role Index

- **HHAeXchange – Senior Data Engineer** 13 stories |p. 2
- **GrowData Analytics – Principal ML Engineer** 9 stories |p. 6
- **Adonis – Senior Data Engineer** 8 stories |p. 11
- **Arkos Health – Senior Data Analyst** 20 stories |p. 14
- **Merative – Senior Data Scientist** 7 stories |p. 19
- **IBM – Senior Data Scientist** 13 stories |p. 22
- **IBM – Senior Data Architect** 11 stories |p. 26
- **IBM – Data Architect** 10 stories |p. 30
- **IBM – Software Developer, Data Platform** 6 stories |p. 33
- **IBM – Data Scientist** 5 stories |p. 35
- **Explorys, an IBM Company – Data Scientist** 3 stories |p. 37
- **AmTrust Financial Services – Software Engineer** 9 stories |p. 38
- **PPG Industries – Color Scientist** 13 stories |p. 41

HHAeXchange – Senior Data Engineer

Dates: Oct. 2025 - Jun. 2026
Technologies: Snowflake, dbt, SQL/Jinja, Python, Snowpark, pandas, Liquibase, GitHub Actions, FHIR/Aidbox

Brought in to unify two distinct data environments following the HHAeXchange-Sandata merger. Built shared Snowflake/dbt models and onboarding patterns, then identified and remediated data-quality defects affecting hundreds of millions of EVV records. Led healthcare source analysis, data-quality investigation and remediation, and Snowflake modeling to produce trusted, standardized datasets for reporting and downstream use. Contract via Veersa Technologies.

hhax-01

Engineered unified and analytics Snowflake/dbt models spanning 55 tables across two organizations and two source-system configurations, establishing reusable patterns for faster client onboarding.

S After HHAeXchange and Sandata came together, the reporting layer had two versions of the same healthcare world. Patients, caregivers, offices, and visits existed in both systems, but their keys, types, and lifecycle rules disagreed. Maintaining parallel analytics would have doubled support effort and made every new report a reconciliation exercise.

T I needed to create one dependable analytical contract while keeping the existing HHA reporting path stable.

A I first mapped where the systems agreed and where they genuinely differed. I preserved source identifiers for traceability, introduced stable warehouse identities, normalized types and lifecycle behavior, and isolated source-specific logic before bringing both paths into common models. I validated each domain with paired examples so downstream teams could see that “unified” did not mean erasing meaningful differences.

R Analysts could work from one coherent model instead of learning two platforms, and the team gained a reusable integration pattern rather than a merger-specific patchwork.

hhax-11

Diagnosed and remediated a multi-payer EVV dashboard data-loss defect by tracing fact-to-dimension joins across 1.12 billion visit records; quantified 214.5 million records (19.2%) excluded by missing patient, service-code, and payer mappings, isolated 99.2% of in-scope missing-patient visits to the CareSource feed, and corrected the underlying integration defects.

S A multi-payer EVV dashboard was visibly undercounting visits, and the first suspicion was a reporting filter. Changing the dashboard would have hidden the symptom without proving whether the warehouse had already discarded the records.

T I needed to find exactly where the population disappeared and determine whether the problem was broad or concentrated in one integration.

A I treated the join path as a funnel, measuring which visits survived each patient, service-code, and payer relationship. That showed roughly one in five visits falling out before the dashboard ever saw them. I then narrowed the missing-patient cohort by payer and source and found that it was overwhelmingly concentrated in the CareSource feed. I traced representative records back to the faulty mappings and corrected the patient, service-code, and payer integration logic.

R We fixed the data loss at its source rather than disguising it in the dashboard, restored confidence in the multi-payer view, and gained a reusable method for investigating future join loss.

hhax-12

Executed structural, row-level, transformation, and completeness QA for a 156-field Snowflake/dbt EVV visit model; uncovered and corrected defects associated with a 52.23% source-to-target row-count gap, 73.6% null patient identifiers, and 105.6 million orphaned visit relationships.

S A new Sandata visit model completed successfully and looked structurally plausible, but downstream analytics would depend on every visit retaining its patient and business context. A green dbt run was not evidence that the population had survived.

T I needed to prove the model’s grain, transformations, and completeness before anyone built reporting on top of it.

A I worked from the outside in: verified the schema and field behavior, tested visit identity, reconciled the source population, and then followed relationships into the patient model. The turning point came when the reconciliation showed that more than half of the expected visits were missing and that null patient identities and orphaned relationships were systemic, not edge cases. I traced the losses through the join and mapping logic, corrected the responsible transformations, and reran the same acceptance sequence.

R The team avoided publishing a model that looked complete but was analytically unsafe, and the repaired visit layer moved forward with a reproducible body of evidence behind it.

hhax-13

Validated and remediated an EVV patient-address model, resolving six documented defects involving deletion handling, multi-address coverage, usage-tag logic, and production data types; verified 1.31 million unique keys, zero missing parent references, and exact 158,084-row parity for the deduplicated parent-matched population.

S The first patient-address model appeared workable for a single address, but real patients could have multiple addresses with different purposes and lifecycle states. The model also failed in production when an upstream type changed, revealing that both its business logic and operational resilience needed attention.

T I needed to redesign the model around the patient relationship, correct the address semantics, and prove that the production fix did not introduce new data loss.

A I documented each failure mode, reused the governed patient entity, incorporated the appropriate supplemental address sources, added deletion and usage semantics, aligned the key pattern with the established HHA model, and delivered a focused type-handling hotfix. I then validated uniqueness, parent linkage, required fields, join coverage, filters, and source-to-target reconciliation as one acceptance path.

R The model supported real multi-address behavior, every retained address resolved to a patient, the reconciled population matched its source exactly, and the production issue was closed without sacrificing analytical correctness.

hhax-05

Conducted feasibility analysis of source-system structures and implementation constraints to shape integration scope, modeling strategy, and delivery approach.

S The team was under pressure to commit to a unified EVV design before anyone had proven that the Sandata snapshots contained the history and relationships the existing analytics required. The risk was promising parity and discovering late that the source could not support it.

T I needed to separate what was genuinely feasible from what required a changed scope or an explicit limitation.

A I treated the source like an investigation rather than a specification. I profiled raw behavior, followed candidate joins, challenged assumed grains, and tested how updates and deletions appeared over time. I brought the gaps back as concrete examples and used them to shape staging boundaries, identity rules, and acceptance criteria before the full build began.

R Stakeholders made scope decisions with evidence, engineering avoided designing around false assumptions, and known source limitations became planned constraints instead of late surprises.

hhax-04

Translated ambiguous business requirements into documented mappings, acceptance criteria, onboarding standards, and 12 data-quality investigation runbooks.

S Requirements often arrived as a simple request to make Sandata data “work like HHA.” That phrase concealed important choices about grain, joins, lifecycle states, and how much variance was acceptable. Different reviewers could approve different behavior while believing they agreed.

T I needed to turn an ambiguous comparison into a shared definition of done.

A I translated each business expectation into a mapping, an acceptance criterion, and an executable investigation path. The resulting runbooks led an engineer from symptom to evidence: what to query, what relationship to expect, how to classify severity, and what had to be retested before sign-off. I used real failure examples so the documents reflected how investigations actually unfolded.

R Reviews became faster and less subjective, disagreements surfaced while they were still design questions, and new team members had a practical playbook for diagnosing future data problems.

hhax-07

Defined surrogate-key, natural-key, and crosswalk strategies supporting stable identity resolution across related source systems.

S Both source systems reused local identifiers. Two unrelated patients or offices could share the same numeric ID simply because they belonged to different accounts, which meant a naive join could create a believable but incorrect relationship.

T I needed identities that were stable across the warehouse without severing the path back to the source record.

A I defined the true business grain for each entity, built composite natural keys with the account and domain context needed to make them meaningful, and mapped those keys to stable warehouse identifiers. I carried the same rules through facts, dimensions, and crosswalks, then deliberately searched for collisions and broken references.

R Shared models stopped confusing locally reused IDs, relationships remained traceable to their origin, and later identity corrections could be made in one governed place.

hhax-06

Structured target models using FHIR-like concepts where appropriate to improve consistency and extensibility for downstream integration and analytics.

S In the merged environment, the same healthcare concept appeared under different names and table structures. Downstream users had to understand each source before they could answer a basic question about a patient, caregiver, location, or address.

T I needed a target model that felt familiar across systems and could evolve without falsely claiming full FHIR conformance.

A I borrowed FHIR-like resource boundaries where they clarified the data, separated core entities from relationships and supporting attributes, and preserved provenance at every boundary. I repeatedly checked the conceptual design against actual report fields and join paths, rejecting abstractions that made the real workflow harder to express.

R The warehouse acquired a more intuitive vocabulary, mappings became easier to review, and future integrations had recognizable extension points instead of another source-shaped schema.

hhax-02

Implemented reusable SQL/Jinja transformation logic for lifecycle states, null handling, sentinel dates, and source-system variations across organization, office, caregiver, and patient domains.

S The most dangerous differences between the HHA and Sandata feeds were not dramatic schema changes; they were small semantic disagreements. A blank value could mean “unknown” in one feed and “not applicable” in another, while an inactive or deleted record could remain apparently current if interpreted literally.

T I needed to make those differences predictable before they reached shared patient, caregiver, office, and organization models.

A I turned recurring edge cases into reusable SQL/Jinja rules for types, nulls, dates, lifecycle states, and deterministic record selection. I kept source labels and application IDs visible, so anyone investigating an exception could follow it back to the originating system rather than treating the shared model as a black box.

R New domains inherited consistent behavior, reviewers could explain why a record was active or excluded, and source variation stopped leaking into every downstream model.

hhax-03

Created or materially redesigned 30 automated SQL/dbt data-quality tests covering nulls, duplicates, referential integrity, business rules, lifecycle logic, and source-to-target reconciliation.

S New EVV models were progressing through environments, but review still depended too heavily on spot checks. A model could compile successfully while quietly duplicating patients, breaking parent-child relationships, or mishandling a deletion state.

T I needed to turn those hidden risks into failures that appeared early and explained themselves.

A I built a layered dbt test suite covering structural expectations, business rules, relationships, lifecycle behavior, and source-to-target reconciliation. When built-in tests could not handle quoted identifiers or source-specific rules, I created reusable generic tests. I also reran the exact failing cohort after each fix so a green result represented regression evidence, not a fresh sample.

R Quality review became a repeatable gate rather than a judgment call, defects reached engineers with a clear failure mode, and releases could proceed with evidence instead of optimism.

hhax-09

Collaborated with client and internal stakeholders to resolve source ambiguity, refine requirements, and translate business needs into production-ready data assets.

S Some of the most important EVV rules could not be discovered from column names. The data alone could not tell us which patient address was authoritative, what a usage tag meant, or which lifecycle event downstream reporting considered final. Guessing would have produced clean-looking but incorrect models.

T I needed to get the right business decision without asking stakeholders to reason from abstract schemas.

A I brought concrete patient and visit examples, showed how each possible interpretation changed the output, and framed the decision in operational terms. Once stakeholders chose the intended behavior, I captured it in mappings, acceptance criteria, and QA evidence, incorporated peer feedback, and demonstrated the result in Snowflake before promotion.

R Ambiguity became an explicit, testable rule, stakeholders understood the consequence of the choice, and production logic had a documented business rationale.

hhax-08

Resolved data-quality and mapping issues through targeted test-case analysis and review of failing source and target records.

S Integration defects rarely announced where they began. A missing dashboard record might originate in a raw source value, a staging cast, a deduplication rule, or a join several layers upstream. Broad fixes risked disturbing data that was already correct.

T I needed to find the first point where a known-good record changed or disappeared and correct only that behavior.

A I built small cohorts around representative failures and followed the same identifiers from raw ingestion through staging, core, and BI. I removed joins one at a time, compared null and population changes, and used the earliest divergence to identify the faulty rule. After making a focused correction, I replayed the original case as a regression test.

R Investigations became explainable rather than speculative, fixes stayed narrowly scoped, and reviewers could see both the defect and the evidence that it was gone.

hhax-10

Established reusable patterns for onboarding additional sources and extending integration workflows.

S The merger was only the immediate integration challenge. If the solution depended on hard-coded exceptions, the next acquired or partner source would force the team to solve the same architectural problem again.

T I needed to leave behind an onboarding path that another engineer could understand and repeat.

A I standardized the movement from staging to core to BI, localized source-specific logic behind a common contract, and made identity fields, provenance, tests, runbooks, and versioned database changes part of the adapter pattern. I wrote the pattern from the perspective of the next engineer: where a new source should vary, what must remain invariant, and what evidence was required before release.

R Future onboarding became an exercise in implementing a known contract rather than redesigning the warehouse, reducing both delivery risk and dependence on the original authors.

GrowData Analytics – Principal ML Engineer

Dates: Sep. 2024 - Oct. 2025
Technologies: Python, Ruby, RubyLLM, RSpec, RuboCop, Tesseract OCR, LLM APIs, Docker, AWS, GitHub Actions

Founded GrowData Analytics to explore how AI led analytics could help small local farms make better operational decisions. The work exposed the more fundamental obstacle that the source data was trapped in inconsistent invoices. I built a preproduction document-to-data product with OCR, validation, audit trails, operational analytics, and a lightweight AWS deployment.

`growdata-01`

Built and validated a preproduction document-to-data MVP that converted semi-structured invoices into auditable transaction records, achieving 95% field-level accuracy across a stratified 500-invoice test suite.

S Small local farms had valuable purchasing and operating data trapped in handwritten and printed invoices whose layouts and image quality varied widely.

T I needed to prove that a document-to-data product could produce useful records without sacrificing accuracy or auditability.

A I assembled a stratified 500-invoice test suite across representative vendors, layouts, and quality levels; combined image preprocessing, OCR, language-model extraction, a versioned JSON schema, and arithmetic reconciliation; and compared outputs field by field while iterating on prompts and post-processing rules.

R The preproduction MVP achieved 95% field-level accuracy and produced structured transaction records that retained enough context for review and correction.

N **Notes / audit prompts**

- **MVP (minimum viable product):** the smallest end-to-end implementation that can test whether the proposed product creates real value.
- **OCR (optical character recognition):** converts pixels in an invoice image into machine-readable text; it is analogous to an ingestion parser for documents.
- **Language-model extraction:** maps noisy OCR text into named business fields such as vendor, date, line item, quantity, and total. The model proposes values; deterministic validation decides whether they are acceptable.
- **Stratified test suite:** a benchmark deliberately sampled across vendors, layouts, handwriting, printing, and image quality so one easy document type cannot dominate the accuracy result.

growdata-02

Instrumented the MVP with automated tests, audit logging, retry controls, and confidence routing, achieving 88% straight-through processing and under 1% system failure.

S High average extraction accuracy was not sufficient for an operational product because uncertain invoices and transient provider failures still needed safe handling.

T I needed to maximize unattended processing while ensuring questionable records were reviewed rather than silently accepted.

A I added automated tests and audit events, bounded retry controls for transient failures, validation and reconciliation signals, and confidence-based routing that separated clean records from invoices requiring human review. I tracked system failures independently from data-quality exceptions so each could be improved appropriately.

R The MVP reached 88% straight-through processing while holding system failure below 1%, with an explicit review path for the remainder.

N **Notes / audit prompts**

- **Straight-through processing (STP):** a document completes ingestion, extraction, validation, and persistence without human intervention.
- **Confidence routing:** combines extraction confidence and deterministic checks to send reliable records forward and questionable ones to a review queue. It is comparable to routing invalid or suspect data into quarantine instead of a trusted table.
- **Bounded retry:** retries a transient failure only a limited number of times, usually with delay or backoff, so a failing dependency cannot create an endless loop.
- **System failure versus data-quality exception:** a timeout or unavailable provider means the pipeline failed to execute; an implausible total or missing vendor means it executed but produced data that should not be accepted automatically.
- **Audit event:** a structured record of what ran, which versions were used, what checks passed, and why the invoice was accepted, retried, or routed for review.

growdata-04

Added schema checks, cross-field validation, reconciliation, idempotent reruns, fixtures, golden-file tests, linting, and release automation.

S Changes to OCR, prompts, or post-processing could improve one invoice while breaking required fields, totals, or previously correct layouts.

T I needed a release gate that protected the transaction contract and made reprocessing safe.

A I enforced a versioned JSON schema, added cross-field and line-to-total reconciliation, designed idempotent reruns, built representative fixtures and golden-file regression tests, and connected tests and linting to release automation.

R Changes could be evaluated against known invoices before release, malformed or inconsistent records were stopped early, and failed work could be rerun without creating duplicate transactions.

N **Notes / audit prompts**

- **Cross-field validation:** checks relationships between fields, such as a due date following an invoice date or a line amount agreeing with quantity times price.
- **Reconciliation:** independently recomputes subtotals, tax, and totals to detect plausible-looking extraction errors.
- **Idempotent rerun:** processing the same invoice again produces or updates the same logical transaction rather than inserting a duplicate.
- **Fixture:** a controlled test input with known characteristics. A **golden file** is the reviewed expected output used for regression comparison.
- **Linting and release automation:** static checks and CI steps that prevent inconsistent code or failed tests from being promoted into a release.

growdata-03

Productized Tesseract OCR and rule-based post-processing as a small Python/Ruby service with containerized deployments and versioned configuration.

S Early OCR experiments worked as isolated scripts but were sensitive to machine setup, image preparation, and unrecorded parameter changes.

T I needed to turn that experimentation into a reproducible component that could be operated and improved like a product.

A I wrapped Tesseract behind an extraction interface, added repeatable image orientation, grayscale, contrast, and normalization steps, applied rule-based cleanup, exposed the workflow through a small Python/Ruby service, and packaged its dependencies and versioned settings in containers.

R The same invoice and configuration produced consistent behavior across environments, making OCR changes easier to test, deploy, and roll back.

N **Notes / audit prompts**

- **Tesseract:** an open-source OCR engine that recognizes characters but does not by itself understand invoice fields or business meaning.
- **Image preprocessing:** deterministic transformations such as rotation, grayscale conversion, contrast adjustment, resizing, and noise reduction that improve the input before recognition.
- **Rule-based post-processing:** ordinary code that normalizes dates, currency, whitespace, vendor spellings, or common OCR substitutions after recognition.
- **Service wrapper:** a stable interface around the OCR workflow so callers do not need to know its command-line arguments, temporary files, or implementation details.
- **Container:** packages the runtime, libraries, OCR binary, and configuration so the same build can run consistently on a laptop, CI worker, or cloud host.
- **Versioned configuration:** preprocessing and OCR settings are reviewed and released like code, making behavioral changes traceable and reversible.

growdata-06

Built a lightweight analytics layer tracking usage, error categories, latency, and throughput to guide feature tuning and cost control.

S Without operational measurements, I could not tell whether a feature improved customer value or merely added latency and model cost.

T I needed enough observability to guide a small product without creating a large data platform.

A I captured usage, latency, throughput, review routing, and categorized error events; summarized them by workflow and configuration version; and compared quality and processing behavior before and after changes.

R The lightweight analytics layer exposed bottlenecks and recurring failure patterns, gave me evidence for feature and configuration tuning, and kept infrastructure and model-spend decisions tied to actual use.

N **Notes / audit prompts**

- **Observability:** structured evidence about what the system is doing, usually through metrics, logs, traces, and categorized events.
- **Latency:** elapsed time to process one request or document.
- **Throughput:** the amount of work completed in a time window, such as invoices per hour.
- **Error taxonomy:** stable categories separating OCR, validation, provider, infrastructure, and user-input failures so trends are actionable.
- **Configuration-version comparison:** the ML equivalent of comparing pipeline releases: quality, cost, and runtime are measured before and after a prompt, model, preprocessing, or rule change.
- **Model spend:** variable cost from external inference calls, tracked per document or workflow so accuracy improvements can be weighed against operating cost.

growdata-09

Led marketing and client acquisition, including creating promotional materials for the business.

S A technically viable invoice parser still needed a clear value proposition and direct feedback from the small farms it was intended to serve.

T I needed to explain the product in practical terms, create interest, and turn outreach into product learning and client opportunities.

A I translated the workflow into benefits such as reduced data entry and better expense visibility, created promotional and demonstration materials, conducted outreach and product conversations, and used objections and requests to refine positioning and the roadmap.

R GrowData established an initial acquisition process and a feedback loop connecting market conversations to product priorities.

N **Notes / audit prompts**

- **Value proposition:** a concise explanation of the customer problem, the benefit delivered, and why the product is preferable to the current process.
- **Client acquisition:** the path from identifying a prospective user through outreach, qualification, demonstration, follow-up, and a decision to pilot or buy.
- **Positioning:** the language and comparison that help a specific audience understand what the product is for; farmers may care more about saved data-entry time and expense visibility than the underlying OCR or model architecture.
- **Feedback loop:** objections, requests, and observed workflow friction are categorized and fed into prioritization rather than treated as isolated conversations.
- **Roadmap:** an ordered set of product outcomes and experiments, not merely a feature list; market evidence helps determine what should be built next.

growdata-07

Managed infrastructure as code for a minimal AWS footprint using S3, Lambda or scheduled jobs, and CloudWatch alerts through Bash/Zsh and Makefile workflows.

S As a self-funded preproduction product, GrowData needed reliable execution and monitoring without the cost or operational burden of a large platform.

T I needed to deploy the pipeline in a minimal, reproducible AWS footprint.

A I managed the infrastructure as code, used S3 for controlled document and result storage, selected Lambda or scheduled execution according to workload needs, configured least-privilege access and CloudWatch alerts, and wrapped common deployment and operating steps in Bash/Zsh and Makefile workflows.

R The service could be recreated and monitored consistently, remained inexpensive at preproduction volume, and avoided manual environment configuration.

N **Notes / audit prompts**

- **IaC (infrastructure as code):** cloud resources and configuration are declared in versioned files or repeatable scripts instead of being created manually.
- **S3 (Simple Storage Service):** AWS object storage used for source images, intermediate artifacts, and results; conceptually similar to a cloud data-lake landing zone.
- **Lambda:** an event-driven AWS compute service suited to short, isolated jobs. Scheduled execution is preferable when work is batch-oriented or exceeds Lambda limits.
- **CloudWatch:** AWS logging, metrics, dashboards, and alerting used to make failures and unusual runtime behavior visible.
- **Least privilege:** each process receives only the permissions it needs, such as access to a specific bucket path rather than broad account access.
- **Makefile and shell workflows:** thin, repeatable operator commands for building, testing, deploying, and inspecting the service.

growdata-05

Documented runbooks and failure modes covering timeouts, low-confidence OCR, and retries.

S The pipeline depended on image quality, local OCR, and external model APIs, so failures ranged from slow requests to readable but low-confidence results.

T I needed predictable recovery procedures that did not depend on remembering how a particular failure had been solved before.

A I cataloged timeout, malformed-response, low-confidence OCR, validation, reconciliation, and exhausted-retry scenarios; documented the relevant logs and checks; and defined when to retry, preprocess again, route to review, reprocess, or escalate.

R Incidents could be classified and handled consistently, limiting blind retries and reducing the chance that an uncertain invoice would enter the transaction dataset.

N **Notes / audit prompts**

- **Runbook:** an operational decision tree describing symptoms, evidence to inspect, recovery steps, escalation conditions, and how to verify resolution.
- **Timeout:** the dependency did not respond within the allowed duration; this is often transient and may justify a bounded retry.
- **Malformed response:** the provider returned content that could not be parsed or did not match the expected schema; retrying unchanged input may not help.
- **Low-confidence OCR:** the text is readable enough to produce candidates but too uncertain for unattended acceptance, so the document should be preprocessed differently or routed to human review.
- **Exhausted retry:** all permitted attempts failed; the work is moved to a visible failure state rather than silently dropped or retried forever.